Unit–2 Assignment: Linear Data Structures

Course: Basics of Data Structures (ETCCBD201)

School: School of Engineering & Technology

Program: B.Sc. (H) Data Science

Student Name: Naina Dhingra

Roll No.: 2501840022

Section: A

Submission Date: 29.03.26

1. Introduction to Linear Data Structures

In this assignment, I implemented a Linear Data Structures Toolkit (LDST) using Python. Linear data structures store data elements in a straight sequence, where each element is connected to its previous and next adjacent elements. The toolkit includes Dynamic Arrays, Singly and Doubly Linked Lists, Stacks, and Queues.

2. Task 1: Dynamic Array Analysis

Amortized Time Complexity of append()

The append(x) operation in a dynamic array adds an element to the end of the structure.

- **Best Case (O(1)):** Most appends are O(1) because there is pre-allocated internal capacity available in the underlying list.
- **Worst Case (O(n)):** When the array is full, the capacity must be doubled. This requires creating a new array and copying all n existing elements to the new memory space.
- **Amortized Conclusion:** While resizing is expensive, it happens infrequently (only when the power of 2 is reached). Therefore, the "averaged out" or amortized cost per operation is **O(1)**.

3. Task 3: Complexity Discussion (Stack & Queue)

The Stack and Queue ADTs were implemented using a Singly Linked List as the underlying storage to ensure dynamic memory allocation.

Operation	Time Complexity	Simple Explanation
Stack: Push	O(1)	Element is added to the head of the list instantly.
Stack: Pop	O(1)	The head node is removed and its value is returned.
Stack: Peek	O(1)	We simply view the data at the head node without removing it.
Queue: Enqueue	O(1)	By maintaining a 'tail' pointer, we add to the end without traversing.
Queue: Dequeue	O(1)	The 'head' pointer allows us to remove the first element instantly.
Queue: Front	O(1)	Direct access to the head element data.

4. Task 4: Application – Balanced Parentheses Checker

The is_balanced(expr) function uses the custom Stack implementation to validate string expressions.

- **Logic:** Every opening bracket ((, {, [) is pushed onto the stack.
- **Validation:** When a closing bracket is encountered, we pop the stack. If the brackets do not match or the stack is empty, the expression is unbalanced.
- **Result:** A string is balanced only if the stack is completely empty after processing the entire string.